LAPORAN PRAKTIKUM TUGAS AKHIR PENGEMBANGAN APLIKASI BERBASIS WEB

Modul 7: CRUD dan Upload File
pada Laravel



Disusun Oleh:

Nama: Dryhus Dzacky Damingtyas.S
NIM: 707012530010
Kelas: D4SIKC-49-04
Mata Kuliah: Pengembangan Aplikasi Berbasis Web

UNIVERSITAS TELKOM
FAKULTAS ILMU TERAPAN
D4 SISTEM INFORMASI KOTACERDAS

Tujuan

Setelah mengikuti praktikum ini mahasiswa diharapkan dapat:

1. Menjelaskan konsep CRUD dan alur upload file pada Laravel.
2. Membuat migration dan model Upload yang terhubung ke tabel users.
3. Memakai resource controller dan resource route untuk tujuh aksi CRUD.
4. Membuat tampilan Blade untuk daftar, unggah, detail, ubah, dan hapus berkas.
5. Menyimpan file ke storage/app/public dan menghapus file lama saat data berubah.
6. Membatasi setiap berkas agar hanya bisa diakses oleh pemiliknya.

Alat dan Bahan

Praktikum ini dikerjakan pada laptop dengan spesifikasi berikut.

Perangkat Keras

- Laptop dengan prosesor 13th Gen Intel Core i7-13620H.
- RAM 16 GB (terbaca 15,7 GB).
- Sistem operasi Windows 11 Home Single Language build 10.0.26200, versi 25H2.

Perangkat Lunak

- XAMPP 8.2.12, berisi Apache 2.4.58, MariaDB 10.4.32, PHP 8.2.12, dan phpMyAdmin 5.2.1.
- Composer 2.10.3 sebagai dependency manager PHP.
- Laravel Framework 12.70.1.
- Laravel Breeze 2.4.2 sebagai starter kit authentication.
- Node.js 24.11.1 dan npm 11.6.2 untuk mengompilasi asset Breeze.
- Vite 7.3.7 dan Tailwind CSS 3.4.19 sebagai build tool dan kerangka tampilan.
- Visual Studio Code sebagai teks editor.
- Google Chrome 155 untuk menjalankan aplikasi di browser.
- Git 2.53.0 untuk version control.
- Database MySQL/MariaDB bernama db_crud_breeze.

C. Langkah-Langkah Praktikum

Project pada praktikum ini adalah aplikasi baru di folder praktikum7 (D:\XAMPP\htdocs\PABW\praktikum7). Nama project saya buat sendiri, bukan crud-breeze seperti contoh modul, supaya sesuai dengan penamaan folder praktikum. Isi aplikasinya mengikuti modul, yaitu pengelolaan berkas dengan Laravel Breeze. Setiap pengguna yang sudah login bisa mengunggah, melihat, mengubah, dan menghapus berkas miliknya sendiri.

C.1 Menyiapkan Project, Database, dan Breeze

Project dibuat dengan composer create-project laravel/laravel praktikum7. Setelah itu Laravel Breeze dipasang memakai composer require laravel/breeze --dev dan php artisan breeze:install blade. Perintah terakhir menyalin route authentication, controller, view register dan login, layout, serta file routes/auth.php ke dalam project.

Database db_crud_breeze saya buat lewat phpMyAdmin. Konfigurasi koneksi ada di file .env dan bagian yang saya ubah terlihat pada gambar berikut.

```
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=db_crud_breeze
DB_USERNAME=root
DB_PASSWORD=
SESSION_DRIVER=database
```

Kode 1. Konfigurasi database pada file .env.

Nilai DB_CONNECTION diisi mysql karena Laravel 12 memakai SQLite secara bawaan. DB_PASSWORD dibiarkan kosong mengikuti XAMPP. SESSION_DRIVER saya ganti menjadi database agar session tersimpan di tabel sessions, bukan di folder storage.

Setelah konfigurasi database benar, saya menjalankan php artisan migrate. Perintah itu membuat tabel users, password_reset_tokens, sessions, cache, cache_locks, jobs, job_batches, dan failed_jobs.

Front-end Breeze dipasang dengan npm install, lalu dijalankan dengan npm run dev. PowerShell di laptop ini menolak menjalankan npm.ps1 karena execution policy, jadi saya memakai npm.cmd. Vite berjalan di port 5173 dan terminalnya saya biarkan terbuka selama pengujian.

Langkah terakhir pada tahap persiapan adalah php artisan storage:link. Perintah itu membuat public/storage sebagai junction ke storage/app/public. Tanpa junction tersebut, file yang sudah diunggah tidak bisa dibuka lewat browser.

C.2 Migration dan Model Upload

Model dan migration dibuat sekaligus dengan php artisan make:model Upload -m. Hasilnya dua file, yaitu app/Models/Upload.php dan satu file migration di folder database/migrations. Isi method up() saya sesuaikan seperti ini.

```php
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    public function up(): void
    {
        Schema::create('uploads', function (Blueprint $table) {
            $table->id();
            $table->foreignId('user_id')->constrained()->cascadeOnDelete();

            $table->string('judul');
            $table->text('keterangan')->nullable();

            $table->string('filename');
            $table->string('filepath');

            $table->timestamps();
        });
    }

    public function down(): void
    {
        Schema::dropIfExists('uploads');
    }
};
```

Kode 2. Migration tabel uploads.

Kolom user_id memakai foreignId('user_id')->constrained()->cascadeOnDelete(). Setiap berkas jadi terikat pada satu pengguna, dan baris berkas ikut terhapus bila akun pemiliknya dihapus. Kolom keterangan memakai nullable() karena boleh dikosongkan. filename menyimpan nama asli file dari komputer pengguna, sedangkan filepath menyimpan path hasil store(), misalnya uploads/Xy12AbCd.pdf.

Setelah php artisan migrate dijalankan, tabel uploads muncul di database. Model Upload kemudian saya isi seperti berikut.

```php
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Model;

class Upload extends Model
{
    protected $table = 'uploads';

    protected $fillable = [
        'user_id',
        'judul',
        'keterangan',
        'filename',
        'filepath',
    ];

    public function user()
    {
        return $this->belongsTo(User::class);
    }
}
```

Kode 3. Model Upload.

$fillable menentukan kolom yang boleh diisi lewat create() dan update(). Relasi user() memakai belongsTo supaya nama pemilik bisa diambil dengan $upload->user->name.

C.3 Resource Controller dan Route

Controller dibuat dengan php artisan make:controller UploadController --resource. Opsi --resource menyiapkan tujuh method kosong: index, create, store, show, edit, update, dan destroy. Ketujuhnya saya isi karena semuanya dipakai oleh halaman uploads.

Method index dan create ada pada potongan pertama.

```php
public function index()
{
    $uploads = Upload::where('user_id', Auth::id())
        ->latest()
        ->paginate(5);

    return view('uploads.index', compact('uploads'));
}

public function create()
{
    return view('uploads.create');
}
```

Kode 4. UploadController bagian 1 (index dan create).

index hanya mengambil berkas milik pengguna yang sedang login. Filter where('user_id', Auth::id()) membuat berkas pengguna lain tidak pernah masuk ke daftar. latest() mengurutkan dari unggahan terbaru, sedangkan paginate(5) membatasi lima baris per halaman. create() cukup mengembalikan view form.

Method store menangani validasi dan penyimpanan file.

```php
public function store(Request $request)
{
    $request->validate([
        'judul' => 'required|string|max:255',
        'keterangan' => 'nullable|string',
        'file' => 'required|file|mimes:jpg,jpeg,png,pdf,doc,docx|max:2048',
    ]);

    $file = $request->file('file');
    $path = $file->store('uploads', 'public');

    Upload::create([
        'user_id' => Auth::id(),
        'judul' => $request->judul,
        'keterangan' => $request->keterangan,
        'filename' => $file->getClientOriginalName(),
        'filepath' => $path,
    ]);

    return redirect()->route('uploads.index')
        ->with('success', 'Berkas berhasil diunggah.');
}
```

Kode 5. UploadController bagian 2 (store).

Validasi membatasi judul maksimal 255 karakter dan file pada format jpg, jpeg, png, pdf, doc, atau docx dengan ukuran maksimal 2048 KB. File disimpan ke disk public lewat store('uploads', 'public'), sehingga isinya berada di storage/app/public/uploads dengan nama acak. Nama asli tetap dicatat di kolom filename agar pengguna masih mengenali berkasnya.

Method show dan edit memakai route model binding.

```php
public function show(Upload $upload)
{
    abort_if($upload->user_id !== Auth::id(), 403);

    return view('uploads.show', compact('upload'));
}

public function edit(Upload $upload)
{
    abort_if($upload->user_id !== Auth::id(), 403);

    return view('uploads.edit', compact('upload'));
}
```

Kode 6. UploadController bagian 3 (show dan edit).

Parameter Upload $upload membuat Laravel mencari baris berdasarkan id di URL, jadi saya tidak perlu menulis Upload::find($id). Sebelum view dibuka, abort_if memeriksa apakah user_id berkas sama dengan Auth::id(). Kalau berbeda, aplikasi mengembalikan 403 Forbidden. Pemeriksaan yang sama saya ulang di update dan destroy karena keduanya juga bisa dipanggil langsung lewat URL.

Method update dan destroy menangani perubahan dan penghapusan.

```php
public function update(Request $request, Upload $upload)
{
    abort_if($upload->user_id !== Auth::id(), 403);

    $request->validate([
        'judul' => 'required|string|max:255',
        'keterangan' => 'nullable|string',
        'file' => 'nullable|file|mimes:jpg,jpeg,png,pdf,doc,docx|max:2048',
    ]);

    $data = $request->only('judul', 'keterangan');

    if ($request->hasFile('file')) {
        Storage::disk('public')->delete($upload->filepath);

        $file = $request->file('file');
        $data['filename'] = $file->getClientOriginalName();
        $data['filepath'] = $file->store('uploads', 'public');
    }

    $upload->update($data);

    return redirect()->route('uploads.index')
        ->with('success', 'Berkas berhasil diperbarui.');
}

public function destroy(Upload $upload)
{
    abort_if($upload->user_id !== Auth::id(), 403);

    Storage::disk('public')->delete($upload->filepath);
    $upload->delete();

    return redirect()->route('uploads.index')
        ->with('success', 'Berkas berhasil dihapus.');
}
```

Kode 7. UploadController bagian 4 (update dan destroy).

File baru pada update bersifat opsional. Kalau pengguna memilih file, file lama dihapus dari storage lebih dulu supaya tidak ada file yang tertinggal tanpa baris di database. destroy melakukan hal yang sama sebelum baris di tabel uploads dihapus. Keduanya kembali ke halaman daftar dengan pesan sukses di session.

Route resource didaftarkan di routes/web.php.

```php
<?php

use App\Http\Controllers\ProfileController;
use App\Http\Controllers\UploadController;
use Illuminate\Support\Facades\Route;

Route::get('/', function () {
    return view('welcome');
});

Route::get('/dashboard', function () {
    return view('dashboard');
})->middleware(['auth', 'verified'])->name('dashboard');

Route::middleware('auth')->group(function () {
    Route::get('/profile', [ProfileController::class, 'edit'])
        ->name('profile.edit');

    Route::patch('/profile', [ProfileController::class, 'update'])
        ->name('profile.update');

    Route::delete('/profile', [ProfileController::class, 'destroy'])
        ->name('profile.destroy');

    Route::resource('uploads', UploadController::class);
});

require __DIR__.'/auth.php';
```

Kode 8. Route resource uploads di dalam middleware auth.

Route::resource('uploads', UploadController::class) membuat tujuh route sekaligus. Karena diletakkan di dalam Route::middleware('auth')->group(), pengguna yang belum login dan membuka /uploads akan diarahkan ke halaman login. Saya memeriksa hasilnya dengan php artisan route:list --name=uploads. Ketujuh route uploads muncul, mulai dari uploads.index sampai uploads.destroy, semuanya memakai middleware auth.

C.4 Tampilan Blade

Semua halaman uploads memakai komponen x-app-layout supaya navigasi, menu profil, dan tombol Log Out dari Breeze ikut tampil. Menu Berkas saya tambahkan di navigation.blade.php tepat di bawah menu Dashboard.

```blade
<x-nav-link :href="route('uploads.index')" :active="request()->routeIs('uploads.*')">
    {{ __('Berkas') }}
</x-nav-link>
```

Kode 9. Menu Berkas pada navigation.blade.php.

Atribut :active membuat menu Berkas bergaris bawah saat salah satu halaman uploads dibuka. Saya hanya menambahkan versi desktop. Versi x-responsive-nav-link untuk menu mobile belum saya tambahkan, jadi menu Berkas tidak muncul kalau navigasi mobile dibuka.

Halaman daftar berkas ada di resources/views/uploads/index.blade.php dan saya bagi menjadi tiga potongan gambar.

Bagian atas halaman memuat judul, pesan sukses, tombol tambah, dan kepala tabel.

```blade
<x-app-layout>
  <x-slot name="header">
    <h2 class="font-semibold text-xl text-gray-800 leading-tight">
      Daftar Berkas
    </h2>
  </x-slot>

  <div class="py-12">
    <div class="max-w-7xl mx-auto sm:px-6 lg:px-8">
      <div class="bg-white shadow-sm sm:rounded-lg p-6">

        {{-- Pesan berhasil dari controller --}}
        @if (session('success'))
          <div class="mb-4 p-3 rounded bg-green-100 text-green-700">
            {{ session('success') }}
          </div>
        @endif

        <a href="{{ route('uploads.create') }}"
         class="inline-block mb-4 px-4 py-2 bg-blue-600 text-white rounded">
          + Unggah Berkas
        </a>

        <table class="w-full border text-left">
          <thead class="bg-gray-100">
            <tr>
              <th class="border p-2">No</th>
              <th class="border p-2">Judul</th>
              <th class="border p-2">Nama File</th>
              <th class="border p-2">Tanggal Unggah</th>
              <th class="border p-2">Aksi</th>
            </tr>
          </thead>

          <tbody>
            @forelse ($uploads as $upload)
              <tr>
                <td class="border p-2">
                  {{ $uploads->firstItem() + $loop->index }}
                </td>

                <td class="border p-2">
                  {{ $upload->judul }}
                </td>
```

Kode 10. index.blade.php bagian 1 dari 3.

Bagian tengah menampilkan baris data beserta aksinya.

```
        <td class="border p-2">
          {{-- Link untuk membuka file di tab baru --}}
          <a href="{{ asset('storage/' . $upload->filepath) }}"
           target="_blank"
           class="text-blue-600 underline">
            {{ $upload->filename }}
          </a>
        </td>

        <td class="border p-2">
          {{ $upload->created_at->format('d-m-Y H:i') }}
        </td>

        <td class="border p-2">
          <a href="{{ route('uploads.show', $upload) }}"
           class="text-sky-600">
            Lihat
          </a>
          |

          <a href="{{ route('uploads.edit', $upload) }}"
           class="text-yellow-600">
            Edit
          </a>
          |

          <form action="{{ route('uploads.destroy', $upload) }}"
             method="POST"
             class="inline">
            @csrf
            @method('DELETE')

            <button type="submit"
                class="text-red-600"
                onclick="return confirm('Yakin ingin menghapus berkas ini?')">
              Hapus
            </button>
          </form>
        </td>
      </tr>
    @empty
      <tr>
        <td colspan="5" class="border p-2 text-center">
          Belum ada berkas yang diunggah.
        </td>
      </tr>
    @endforelse
  </tbody>
</table>
```

Kode 11. index.blade.php bagian 2 dari 3.

Bagian akhir menutup tabel dan menambahkan tautan pagination.

```
            <div class="mt-4">
              {{ $uploads->links() }}
            </div>

          </div>
        </div>
      </div>
    </x-app-layout>
```

Kode 12. index.blade.php bagian 3 dari 3.

Nomor urut memakai $uploads->firstItem() + $loop->index supaya penomoran berlanjut di halaman kedua. Tautan file dibentuk dengan asset('storage/' . $upload->filepath) dan dibuka di tab baru. Tombol Hapus berada di dalam form POST yang diubah menjadi DELETE lewat @method('DELETE'), dengan konfirmasi sebelum dikirim.

Form unggah ada di create.blade.php.

```blade
<x-app-layout>
  <x-slot name="header">
    <h2 class="font-semibold text-xl text-gray-800 leading-tight">
      Unggah Berkas
    </h2>
  </x-slot>

  <div class="py-12">
    <div class="max-w-3xl mx-auto sm:px-6 lg:px-8">
      <div class="bg-white shadow-sm sm:rounded-lg p-6">

        {{-- enctype wajib ada agar file ikut terkirim --}}
        <form action="{{ route('uploads.store') }}"
           method="POST"
           enctype="multipart/form-data">

          @csrf

          <div class="mb-4">
            <x-input-label for="judul" value="Judul" />

            <x-text-input
              id="judul"
              name="judul"
              type="text"
              class="mt-1 block w-full"
              :value="old('judul')"
              required
              autofocus
            />

            <x-input-error
              :messages="$errors->get('judul')"
              class="mt-2"
            />
          </div>

          <div class="mb-4">
            <x-input-label
              for="keterangan"
              value="Keterangan (opsional)"
            />

            <textarea
              id="keterangan"
              name="keterangan"
              rows="3"
              class="mt-1 block w-full border-gray-300 rounded-md shadow-sm"
            >{{ old('keterangan') }}</textarea>
```

Kode 13. create.blade.php bagian 1 dari 2.

```
            <x-input-error
              :messages="$errors->get('keterangan')"
              class="mt-2"
            />
          </div>

          <div class="mb-4">
            <x-input-label
              for="file"
              value="Pilih Berkas (jpg, jpeg, png, pdf, doc, docx - maks. 2 MB)"
            />

            <input
              id="file"
              name="file"
              type="file"
              class="mt-1 block w-full"
              accept=".jpg,.jpeg,.png,.pdf,.doc,.docx"
              required
            >

            <x-input-error
              :messages="$errors->get('file')"
              class="mt-2"
            />
          </div>

          <x-primary-button>
            Simpan
          </x-primary-button>

          <a
            href="{{ route('uploads.index') }}"
            class="ms-3 text-gray-600"
          >
            Batal
          </a>

        </form>
      </div>
    </div>
  </div>
</x-app-layout>
```

Kode 14. create.blade.php bagian 2 dari 2.

Atribut enctype="multipart/form-data" wajib ada supaya file ikut terkirim. Nama input file harus sama dengan yang dipakai $request->file('file') di controller. Komponen x-input-label, x-text-input, x-input-error, dan x-primary-button berasal dari Breeze, dan :value="old('judul')" mengisi kembali input saat validasi gagal.

Form edit hampir sama dengan form create, ditambah @method('PUT') dan informasi file yang sedang dipakai.

```blade
<x-app-layout>
  <x-slot name="header">
    <h2 class="font-semibold text-xl text-gray-800 leading-tight">
      Edit Berkas
    </h2>
  </x-slot>

  <div class="py-12">
    <div class="max-w-3xl mx-auto sm:px-6 lg:px-8">
      <div class="bg-white shadow-sm sm:rounded-lg p-6">

        <form action="{{ route('uploads.update', $upload) }}"
           method="POST"
           enctype="multipart/form-data">

          @csrf
          @method('PUT')

          {{-- Mengubah request menjadi PUT agar menuju method update() --}}

          <div class="mb-4">
            <x-input-label for="judul" value="Judul" />

            <x-text-input
              id="judul"
              name="judul"
              type="text"
              class="mt-1 block w-full"
              :value="old('judul', $upload->judul)"
              required
            />

            <x-input-error
              :messages="$errors->get('judul')"
              class="mt-2"
            />
          </div>

          <div class="mb-4">
            <x-input-label
              for="keterangan"
              value="Keterangan (opsional)"
            />

            <textarea
              id="keterangan"
              name="keterangan"
              rows="3"
              class="mt-1 block w-full border-gray-300 rounded-md shadow-sm"
            >{{ old('keterangan', $upload->keterangan) }}</textarea>
```

Kode 15. edit.blade.php bagian 1 dari 3.

```blade
    <x-input-error
      :messages="$errors->get('keterangan')"
      class="mt-2"
    />
  </div>

  <div class="mb-4">
    <p class="text-sm text-gray-600">
      File saat ini:
      <a
        href="{{ asset('storage/' . $upload->filepath) }}"
        target="_blank"
        class="text-blue-600 underline"
      >
        {{ $upload->filename }}
      </a>
    </p>

    <x-input-label
      for="file"
      value="Ganti Berkas (kosongkan jika tidak ingin mengganti)"
      class="mt-2"
    />

    <input
      id="file"
      name="file"
      type="file"
      class="mt-1 block w-full"
      accept=".jpg,.jpeg,.png,.pdf,.doc,.docx"
    >

    <x-input-error
      :messages="$errors->get('file')"
      class="mt-2"
    />
  </div>

  <x-primary-button>
    Update
  </x-primary-button>

  <a
    href="{{ route('uploads.index') }}"
    class="ms-3 text-gray-600"
  >
    Batal
  </a>
```

Kode 16. edit.blade.php bagian 2 dari 3.



Kode 17. edit.blade.php bagian 3 dari 3.

Kalau pengguna tidak memilih file baru, file lama tetap dipakai. Teks pada label menjelaskan hal itu supaya tidak membingungkan.

Halaman detail menampilkan informasi berkas dan pratinjau.

```blade
<x-app-layout>
  <x-slot name="header">
    <h2 class="font-semibold text-xl text-gray-800 leading-tight">
      Detail Berkas
    </h2>
  </x-slot>

  <div class="py-12">
    <div class="max-w-3xl mx-auto sm:px-6 lg:px-8">
      <div class="bg-white shadow-sm sm:rounded-lg p-6 space-y-2">

        <p>
          <strong>Judul:</strong>
          {{ $upload->judul }}
        </p>

        <p>
          <strong>Keterangan:</strong>
          {{ $upload->keterangan ?? '-' }}
        </p>

        <p>
          <strong>Nama File:</strong>
          {{ $upload->filename }}
        </p>

        <p>
          <strong>Diunggah oleh:</strong>
          {{ $upload->user->name }}
        </p>

        <p>
          <strong>Tanggal Unggah:</strong>
          {{ $upload->created_at->format('d-m-Y H:i') }}
        </p>

        @php
          $ext = strtolower(
            pathinfo($upload->filepath, PATHINFO_EXTENSION)
          );
        @endphp
```

Kode 18. show.blade.php bagian 1 dari 2.

```blade
        {{-- Jika file berupa gambar, tampilkan pratinjau --}}
        @if (in_array($ext, ['jpg', 'jpeg', 'png']))
          <img
            src="{{ asset('storage/' . $upload->filepath) }}"
            alt="{{ $upload->judul }}"
            class="max-w-sm rounded border"
          >
        @else
          <a
            href="{{ asset('storage/' . $upload->filepath) }}"
            target="_blank"
            class="inline-block px-4 py-2 bg-gray-800 text-white rounded"
          >
            Buka / Unduh Berkas
          </a>
        @endif

        <div class="pt-4">
          <a
            href="{{ route('uploads.edit', $upload) }}"
            class="text-yellow-600"
          >
            Edit
          </a>

          |

          <a
            href="{{ route('uploads.index') }}"
            class="text-gray-600"
          >
            Kembali
          </a>
        </div>

      </div>
    </div>
  </div>
</x-app-layout>
```

Kode 19. show.blade.php bagian 2 dari 2.

Pratinjau dibedakan menurut ekstensi file. Gambar ditampilkan dengan tag img, sedangkan PDF dan dokumen lain hanya mendapat tombol Buka / Unduh Berkas. Ekstensi diambil dari path file memakai pathinfo() lalu diubah ke huruf kecil.

C.5 Menjalankan Aplikasi dan Menguji

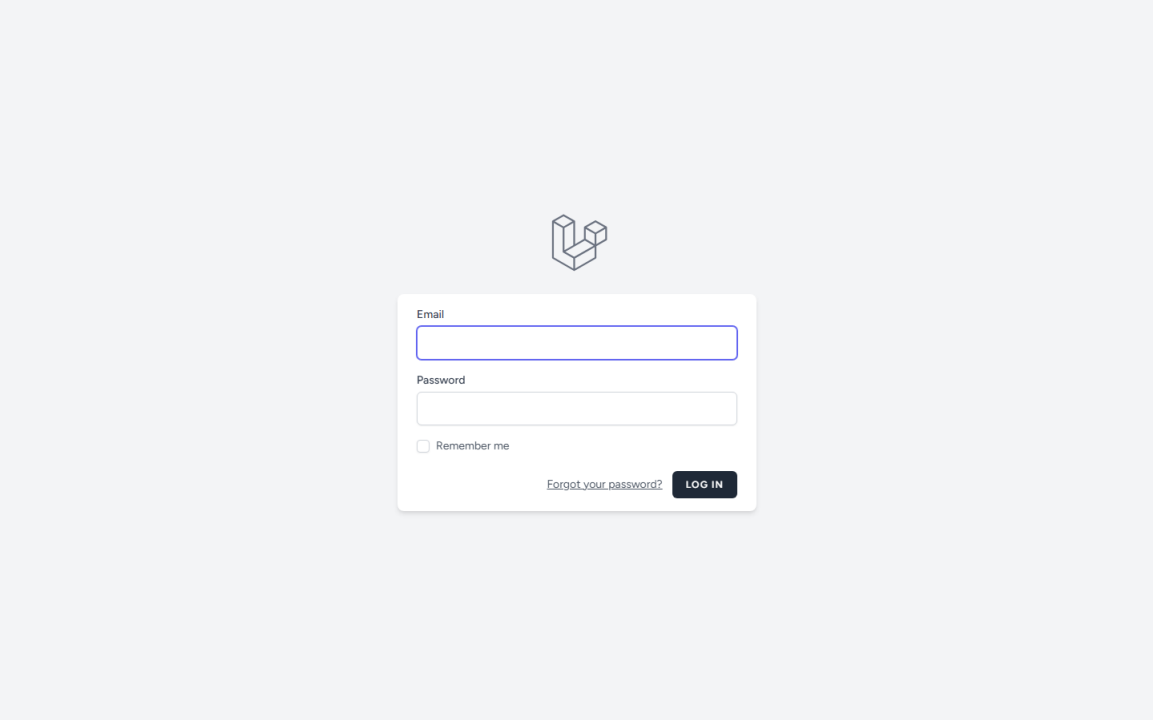
Aplikasi dijalankan dengan php artisan serve. Port 8000 sudah dipakai project lain di laptop ini, jadi saya memakai --port=8123 dan membuka http://127.0.0.1:8123. Vite tetap berjalan di terminal kedua.

Akun uji saya daftarkan lewat halaman register Breeze dengan nama Dryhus Dzacky, email dryhus.modul7@example.com, dan password password123. Akun ini yang dipakai untuk seluruh pengujian CRUD.

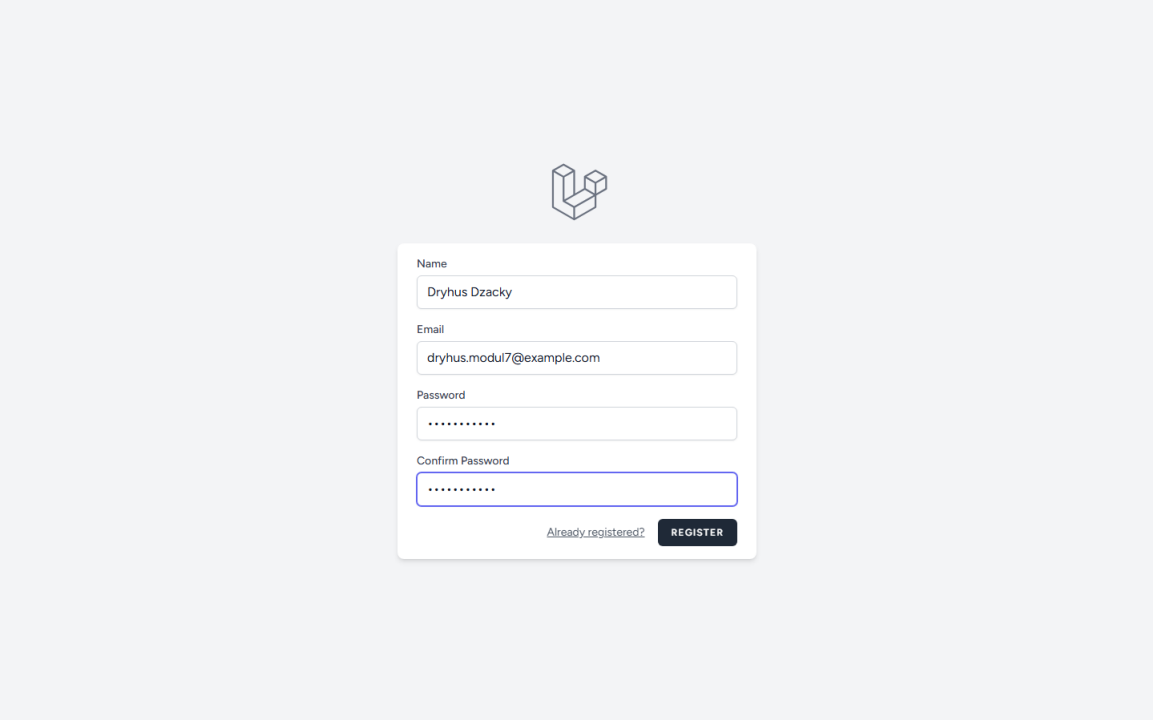
Urutan pengujian mengikuti modul. Saya membuka /uploads sebelum login, mengunggah gambar dan PDF, mencoba file .zip dan file berukuran lebih dari 2 MB, mengganti file lewat form edit, menghapus satu berkas, lalu memeriksa tabel uploads dan folder storage. Hasil tiap langkah ada pada bagian D.

D. Hasil dan Dokumentasi Program

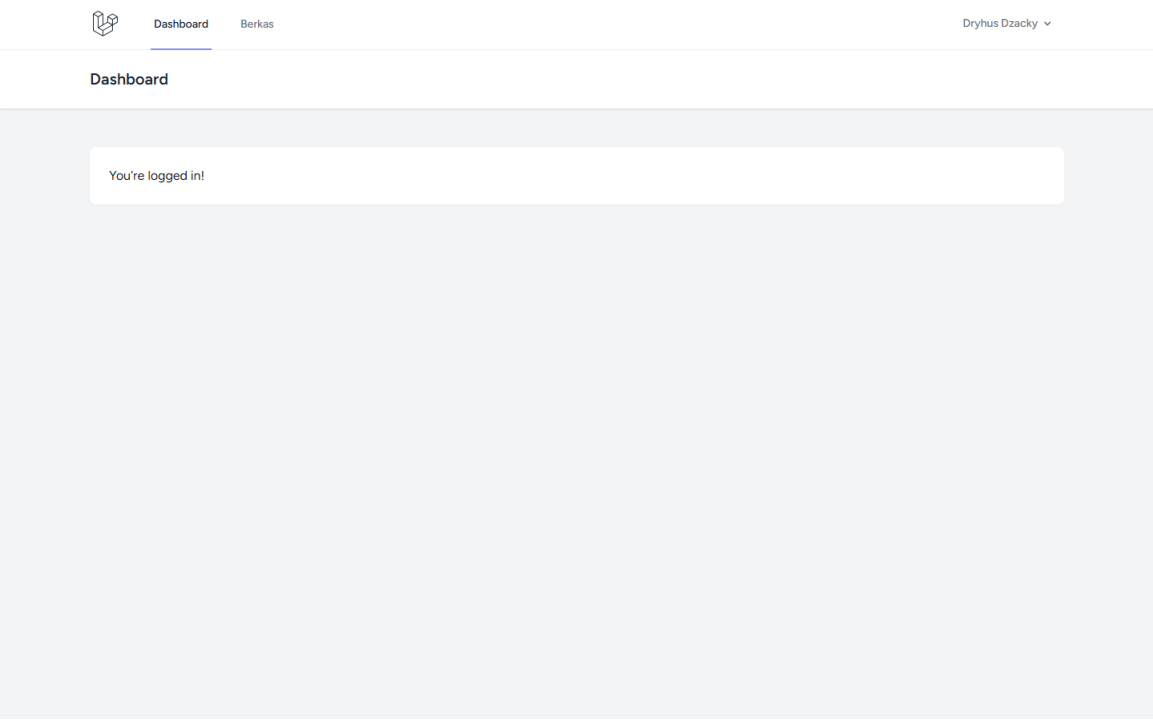
Berikut tangkapan layar dari aplikasi praktikum7 yang sudah dijalankan, ditambah pemeriksaan di phpMyAdmin dan folder storage.



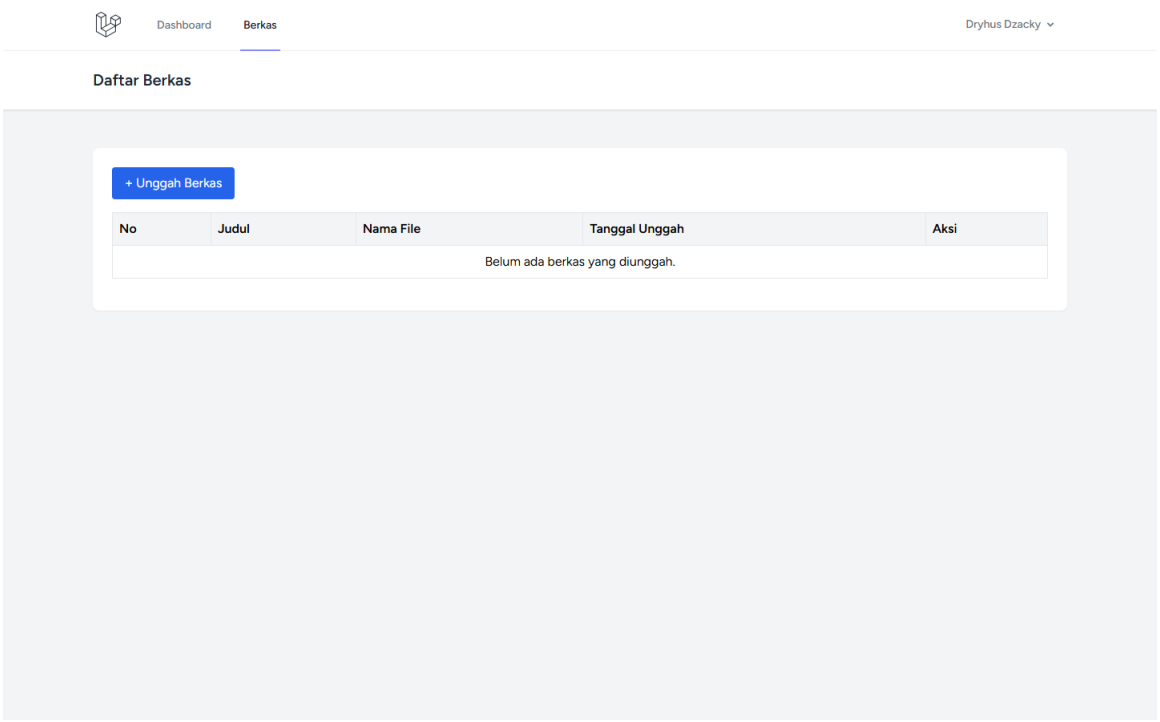
Gambar 1. Halaman login yang muncul saat /uploads dibuka tanpa login.



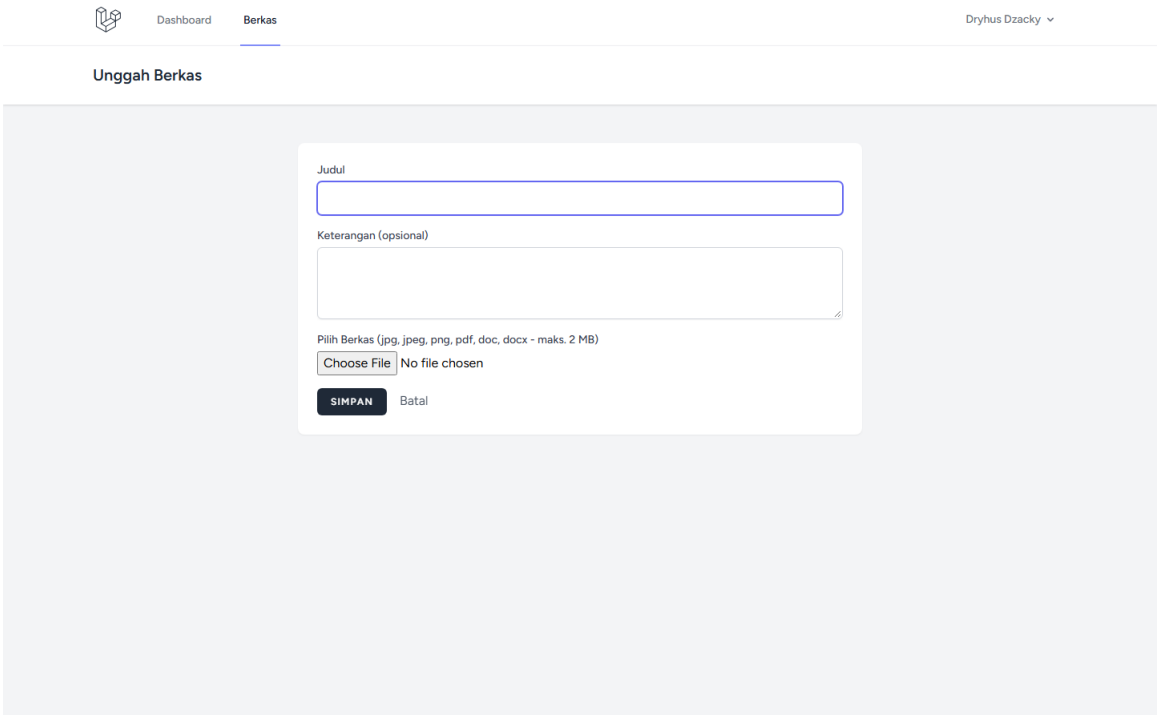
Gambar 2. Form register Breeze yang dipakai untuk membuat akun uji.



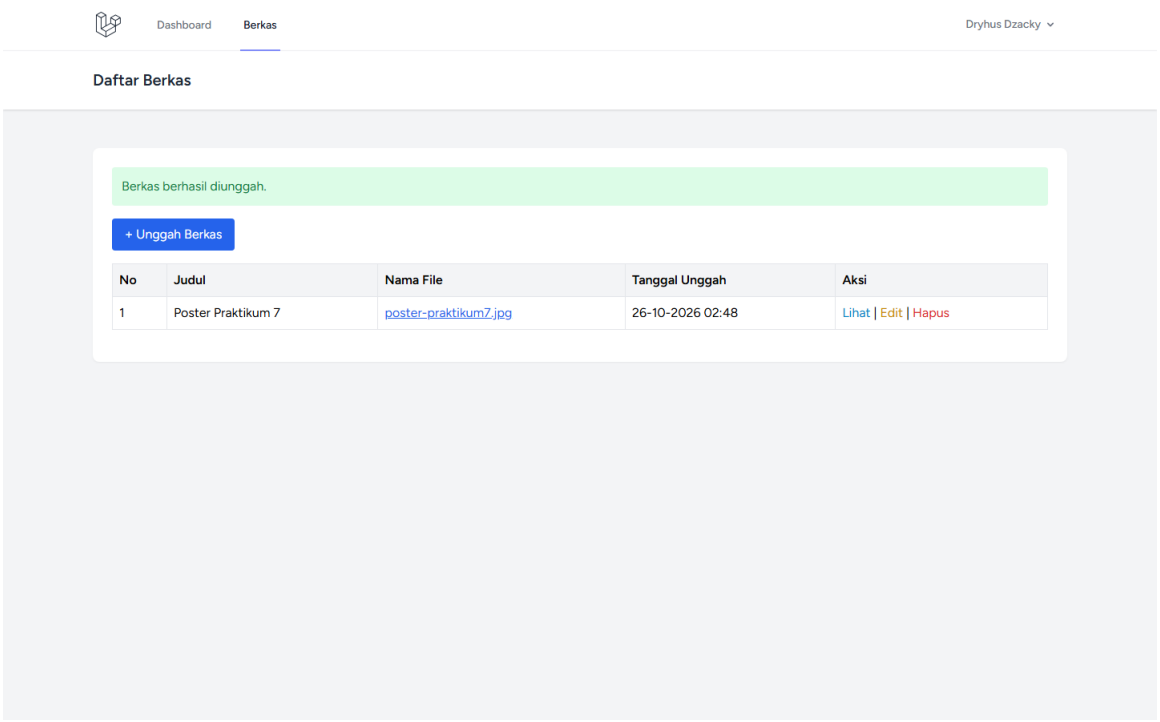
Gambar 3. Dashboard setelah login. Menu Berkas tampil di samping Dashboard.



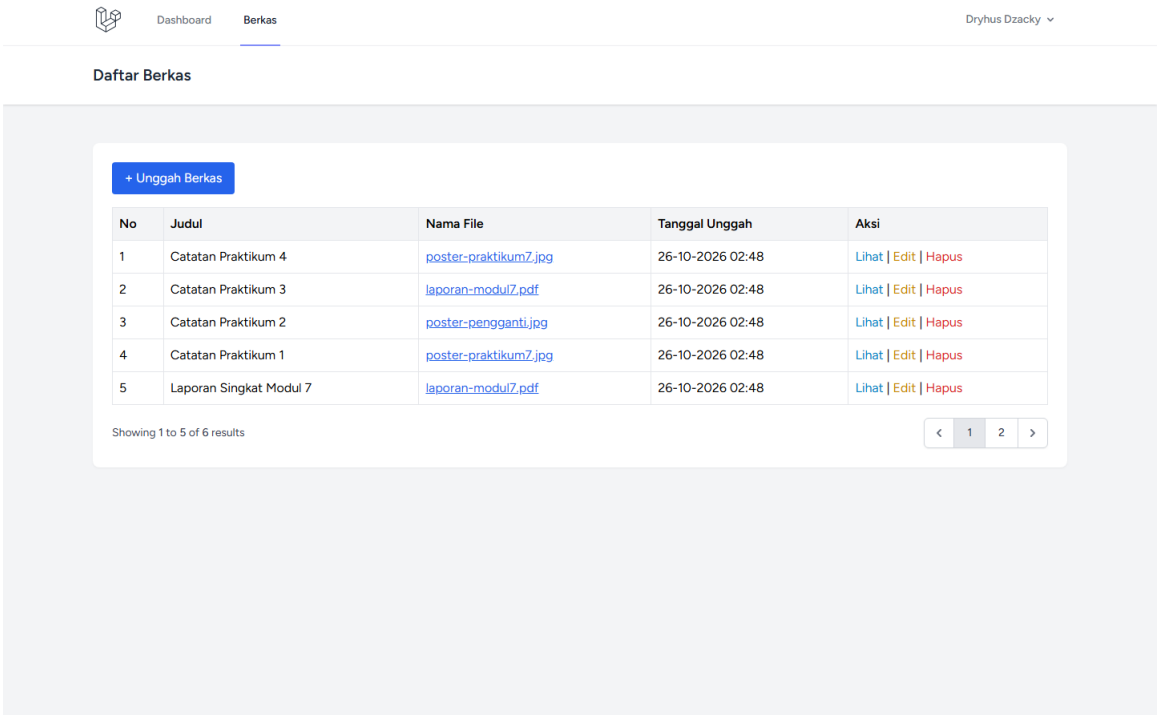
Gambar 4. Halaman daftar berkas saat belum ada data.



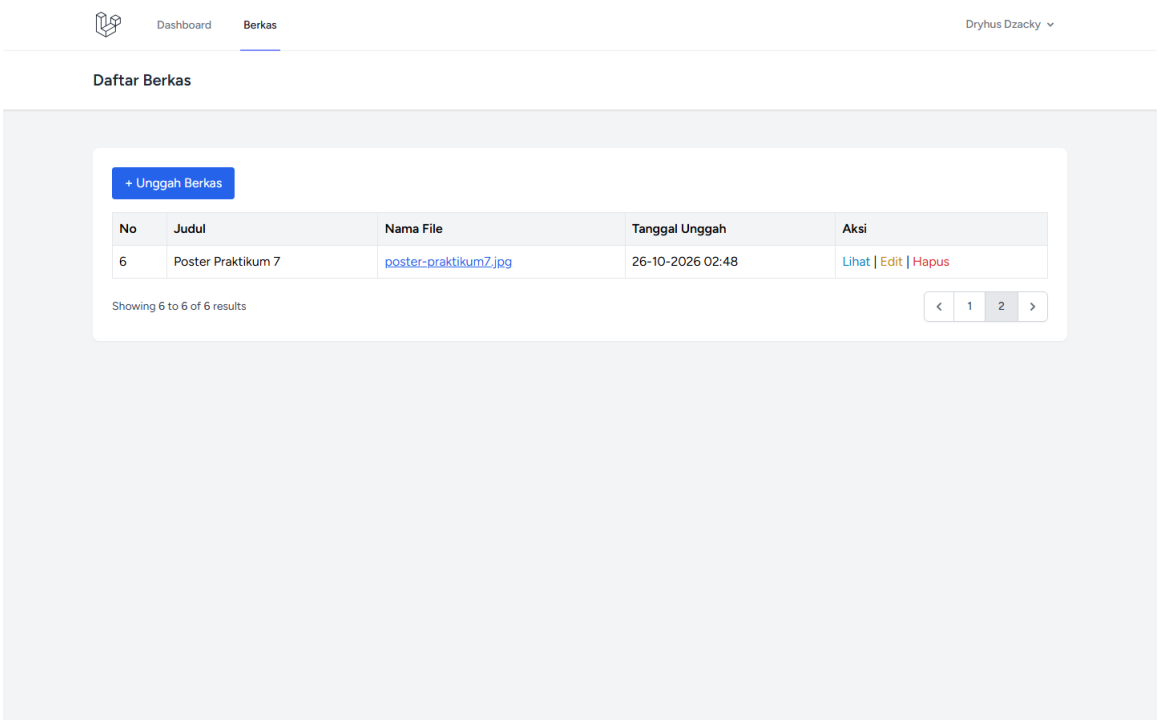
Gambar 5. Form unggah berkas dengan judul, keterangan, dan input file.



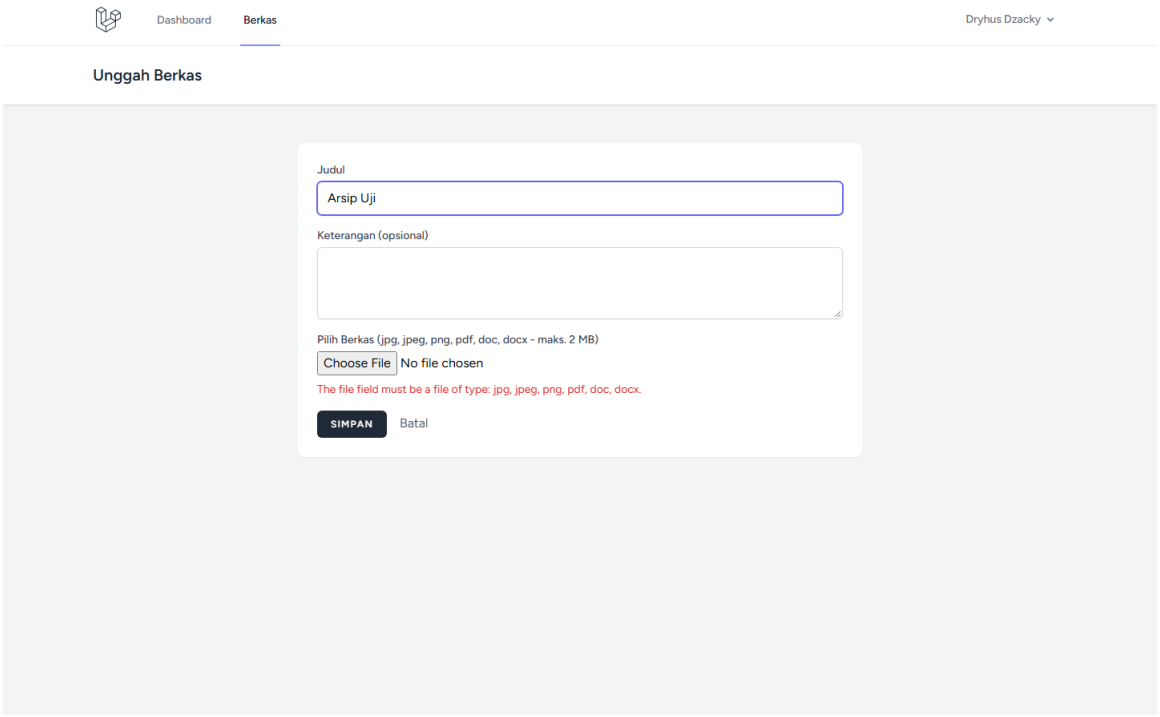
Gambar 6. Pesan sukses setelah berkas pertama diunggah.



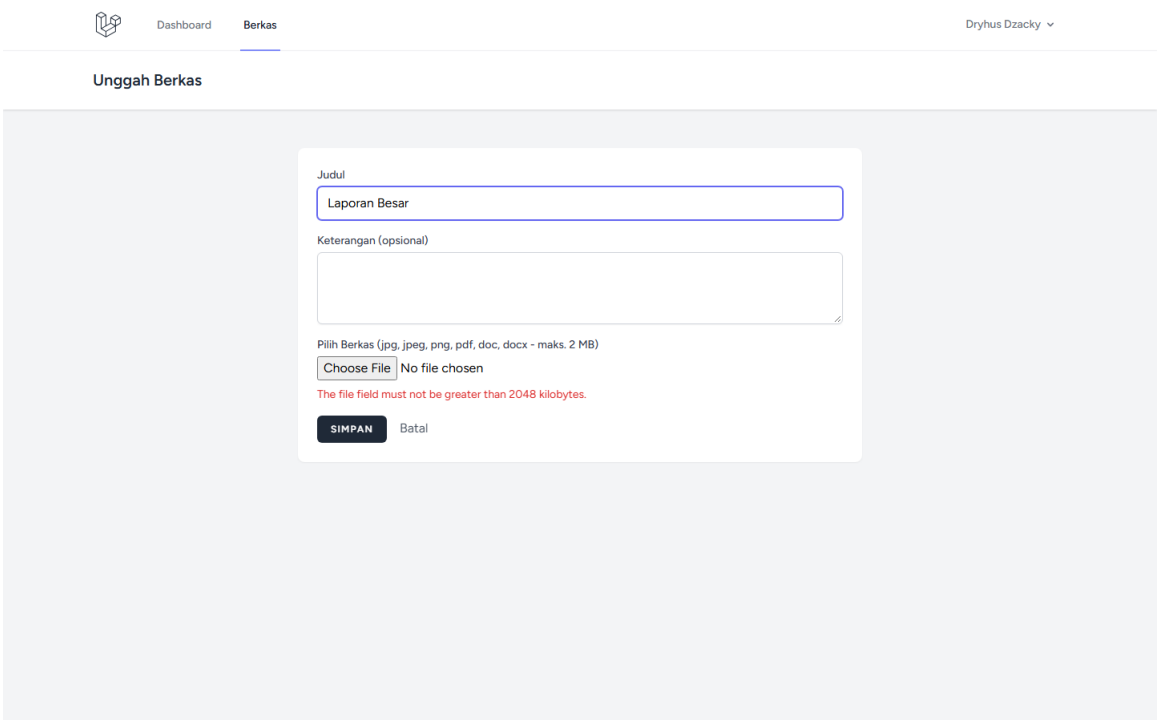
Gambar 7. Halaman pertama daftar berkas berisi lima baris.



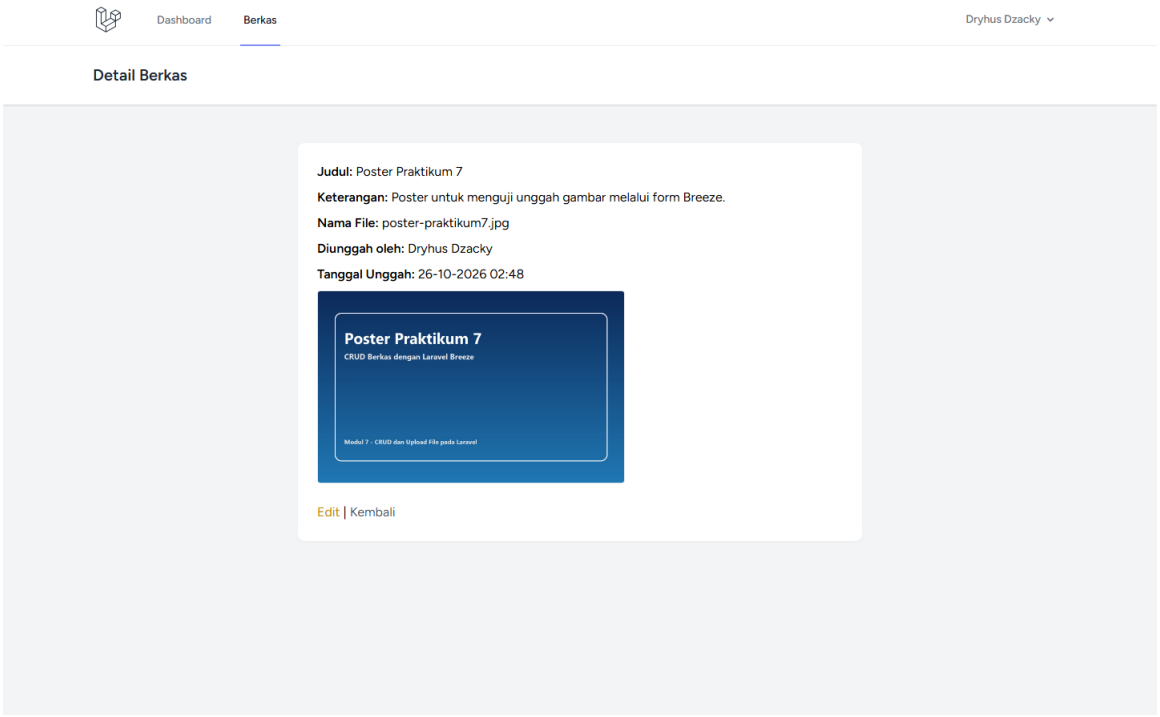
Gambar 8. Halaman kedua daftar berkas. Nomor urut berlanjut dari 6.



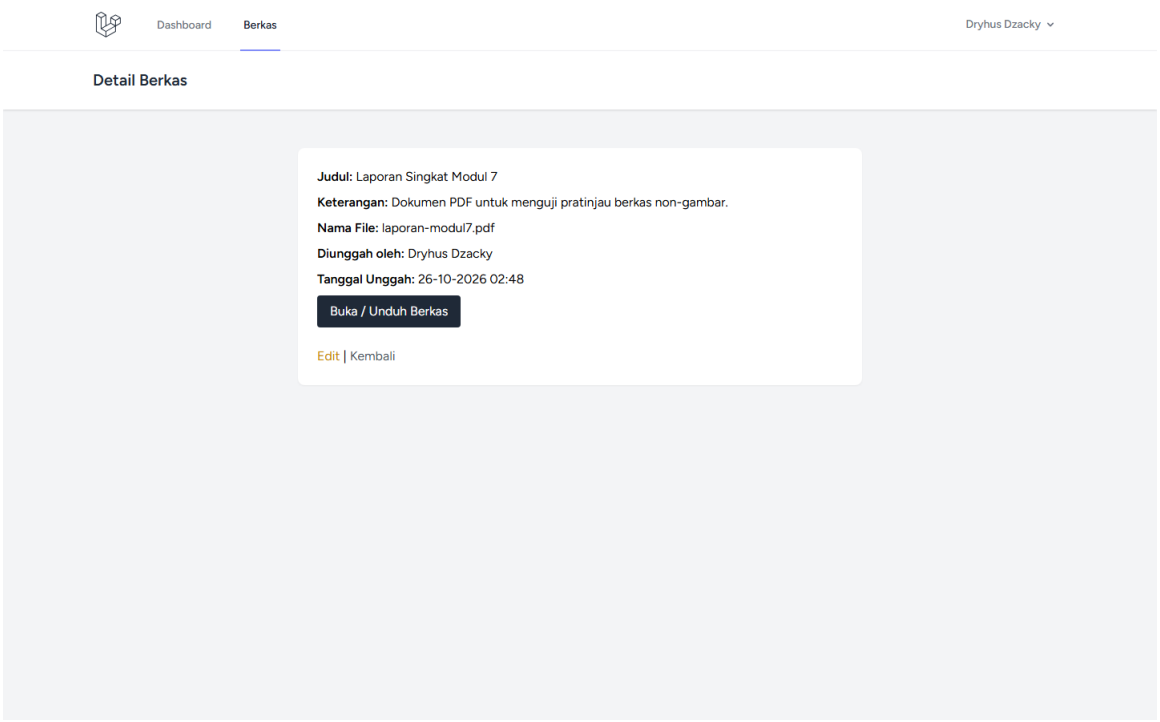
Gambar 9. Galat validasi saat file .zip diunggah.



Gambar 10. Galat validasi saat file melebihi 2 MB.



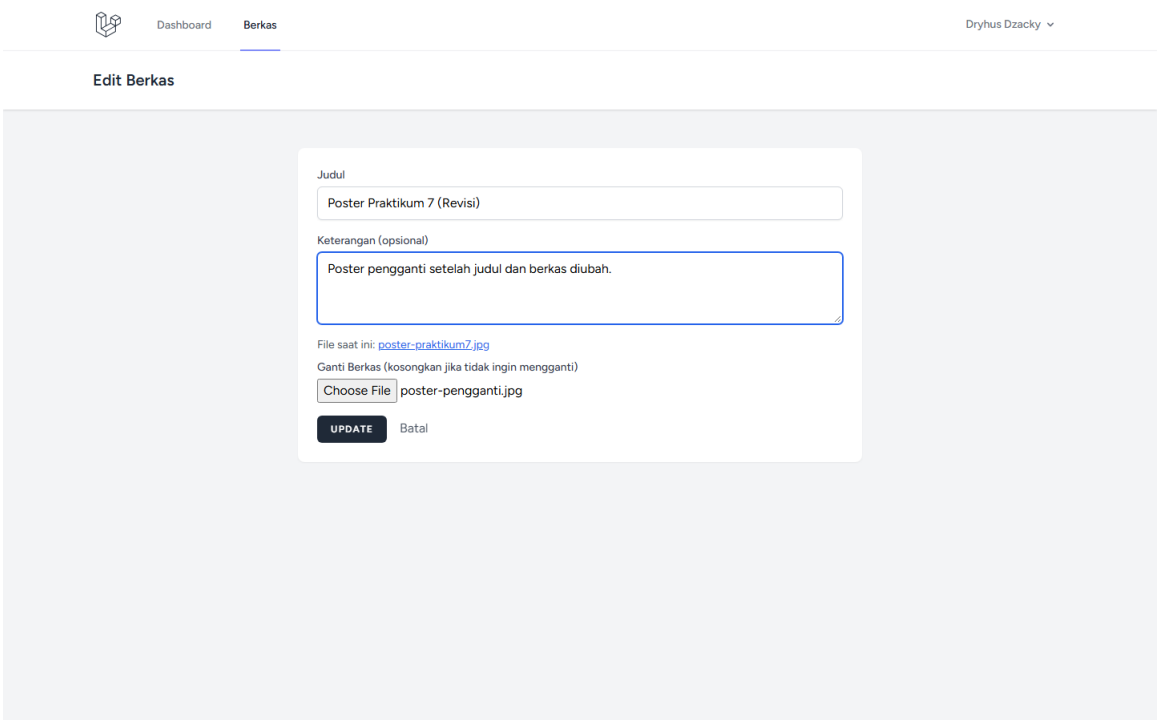
Gambar 11. Halaman detail berkas gambar beserta pratinjaunya.



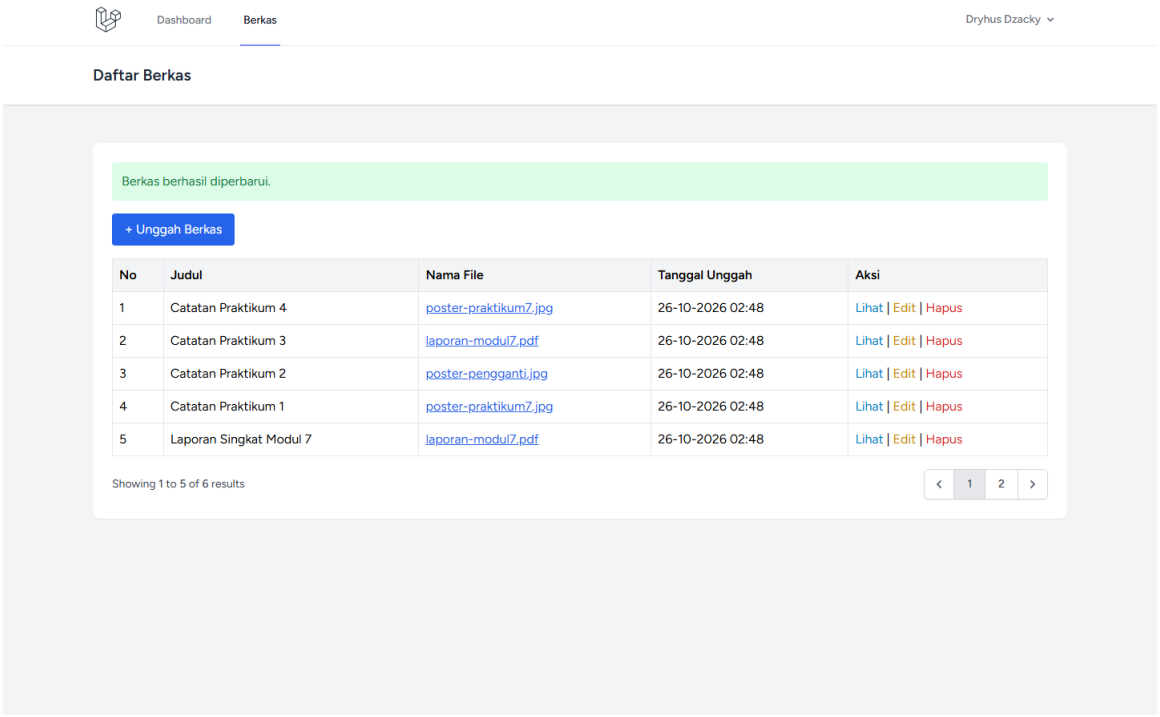
Gambar 12. Halaman detail berkas PDF dengan tombol unduh.



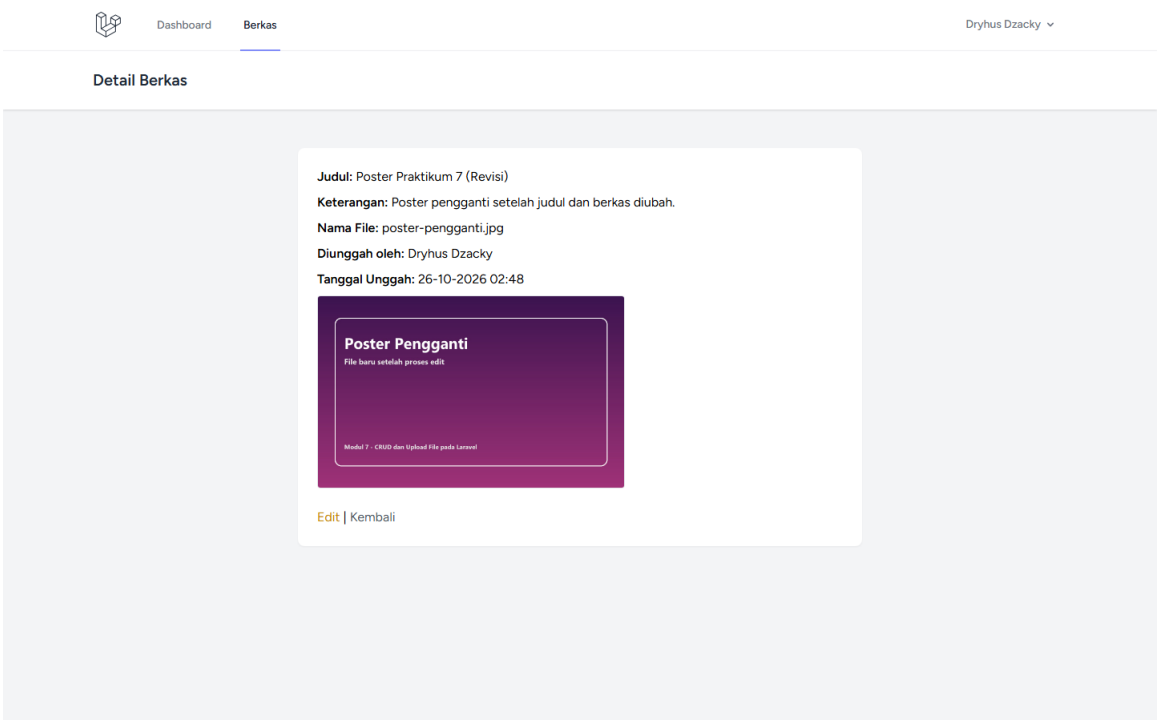
Gambar 13. File yang dibuka lewat URL /storage/uploads/ di tab baru.



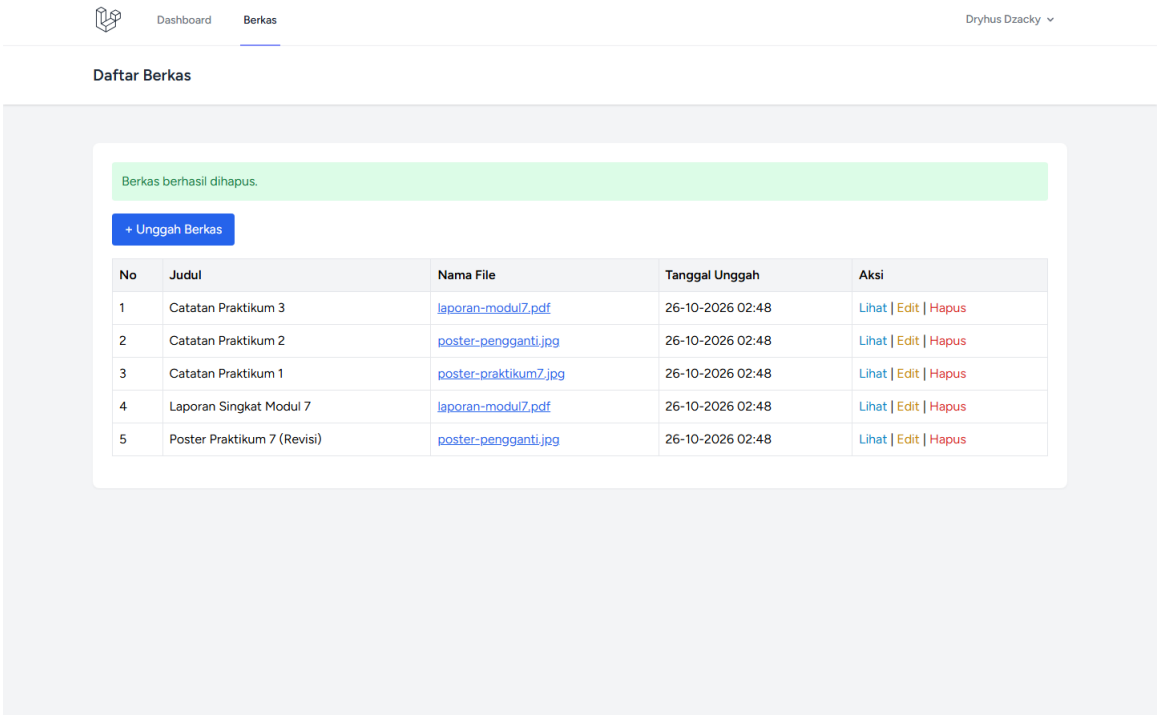
Gambar 14. Form edit dengan judul dan informasi file yang sudah terisi.



Gambar 15. Pesan sukses setelah judul dan file diperbarui.

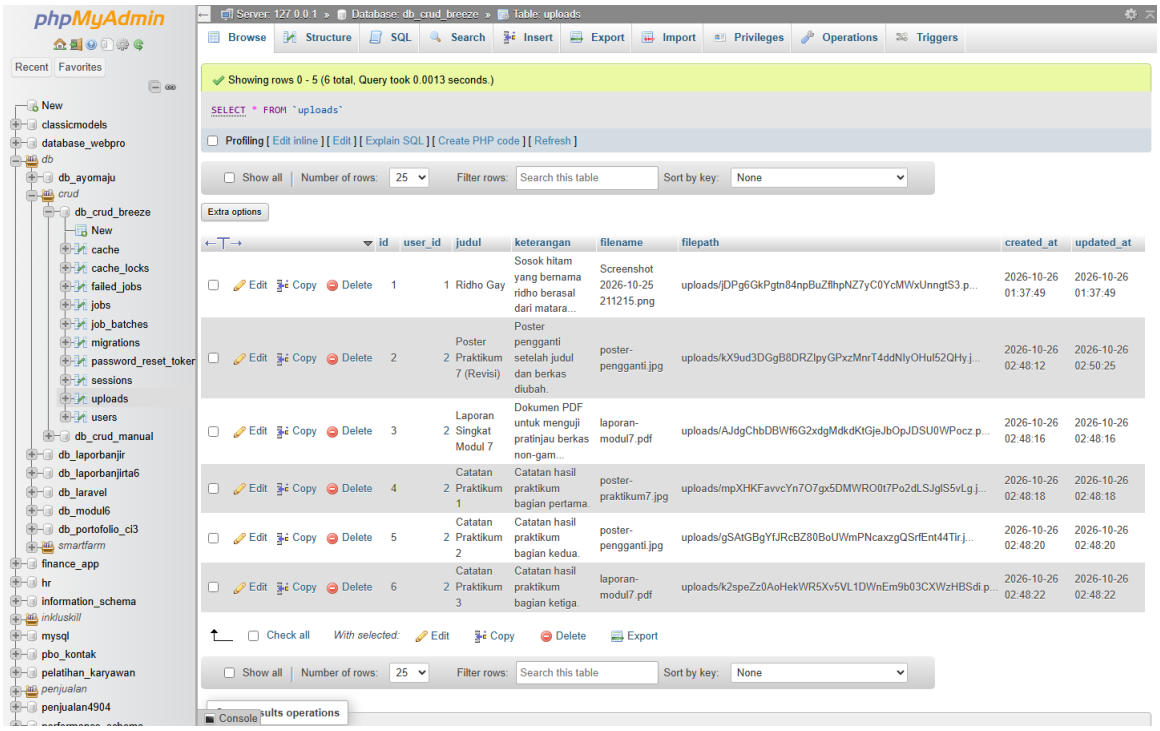


Gambar 16. Halaman detail setelah file diganti. Nama file baru yang tampil.

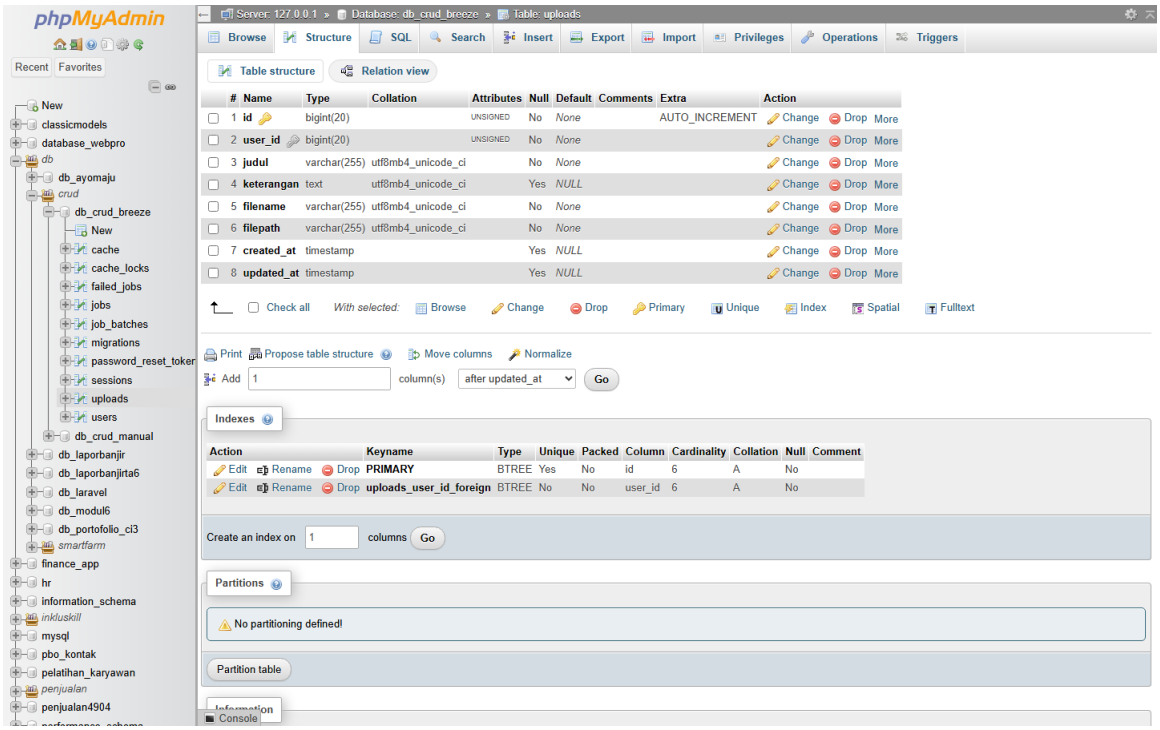


Gambar 17. Pesan sukses setelah berkas dihapus.

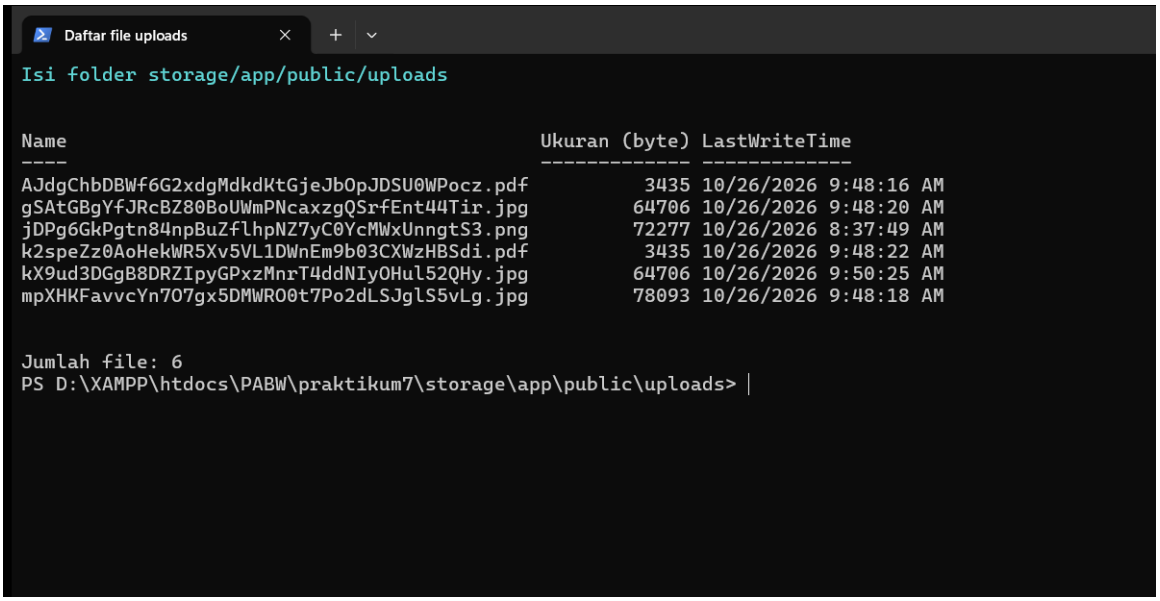
Gambar 18. Halaman 403 saat membuka berkas milik pengguna lain.



Gambar 19. Tabel uploads di phpMyAdmin berisi enam baris.



Gambar 20. Struktur tabel uploads dengan kolom user_id, judul, keterangan, filename, dan filepath.



Gambar 21. Folder storage/app/public/uploads berisi enam file yang sesuai dengan tabel.

E. Analisis dan Pembahasan

E.1 Pemisahan Data per Pengguna

Setiap query pada daftar berkas memakai where('user_id', Auth::id()), jadi pengguna hanya melihat datanya sendiri. Pembatasan di query saja belum cukup karena URL /uploads/1 bisa diketik langsung. Karena itu show, edit, update, dan destroy memeriksa kepemilikan dengan

abort_if. Saya mengujinya memakai akun kedua yang membuka berkas milik akun pertama, dan hasilnya halaman 403 seperti Gambar 18.

E.2 Validasi dan Penyimpanan File

Validasi di controller menangani dua hal, yaitu format dan ukuran file. Format dibatasi lewat mimes, ukuran lewat max:2048. Ketika saya mengunggah file .zip, Laravel kembali ke form dan menampilkan pesan galat pada input file. File 5,7 MB juga ditolak dengan pesan yang sama. Pesan itu berasal dari komponen x-input-error yang membaca $errors->get('file').

File disimpan lewat store('uploads', 'public'). Laravel memberi nama acak supaya dua file dengan nama sama tidak saling menimpa. Nama asli tetap disimpan di kolom filename, sedangkan lokasi file ada di filepath. Pemisahan ini membuat isi file tidak perlu masuk ke database; yang disimpan hanya path-nya.

E.3 Pagination dan Penomoran

index memakai paginate(5). Dengan enam baris data, daftar terbagi menjadi dua halaman. Nomor urut tidak direset karena memakai $uploads->firstItem() + $loop->index, sehingga halaman kedua mulai dari 6. Tautan halaman dibuat oleh $uploads->links().

E.4 Relasi dan Integritas Data

Model Upload memakai belongsTo(User::class), sedangkan kolom user_id memakai cascadeOnDelete(). Relasi dipakai untuk mengambil pemilik berkas, dan cascade menjaga agar tidak ada baris uploads yang menggantung setelah akun dihapus. Ada satu hal yang perlu dicatat: cascade hanya menghapus baris database. File fisik di storage tidak ikut terhapus kalau akun dihapus langsung dari tabel users, karena penghapusan file hanya terjadi di method destroy dan update. Untuk kebutuhan praktikum ini kondisinya masih aman, tetapi aplikasi produksi perlu proses pembersihan tersendiri.

E.5 Keamanan Akses Berkas

Perlindungan berkas terjadi di level aplikasi. Route uploads berada di middleware auth, dan setiap aksi memeriksa kepemilikan data. Namun public/storage adalah junction ke storage/app/public, jadi file bisa dibuka oleh siapa pun yang mengetahui URL-nya tanpa login. Saya membuktikannya dengan membuka URL file di tab baru pada Gambar 13. Kalau berkas bersifat rahasia, file seharusnya tidak diletakkan di disk public dan harus disajikan lewat route yang memeriksa izin.

E.6 Kendala dan Penyelesaian

Ada tiga kendala yang saya temui selama mengerjakan praktikum ini.

1. Perintah npm diblokir kebijakan eksekusi PowerShell. Muncul pesan npm.ps1 cannot be loaded because running scripts is disabled on this system. Saya memakai npm.cmd supaya perintah tetap berjalan.
2. Port 8000 sudah dipakai project praktikum lain, sehingga php artisan serve gagal mengikat port itu. Saya menjalankan server dengan --port=8123 dan menyesuaikan URL pengujian.
3. Saat menjalankan migration saya salah menulis perintah menjadi migrate4. Artisan menolak dengan pesan Command "migrate4" is not defined, dan kesalahan itu tercatat di storage/logs/laravel.log. Setelah perintah ditulis ulang, migration berjalan normal.

E.7 Catatan Perbedaan dengan Modul

Ada beberapa perbedaan antara hasil praktikum ini dan contoh pada modul. Saya mencatatnya supaya pembaca tahu bagian mana yang tidak sama persis.

1. Project diberi nama praktikum7, bukan crud-breeze, agar sesuai dengan folder praktikum.
2. Subbab 7.4.5 tentang CRUD dan upload file tanpa Breeze tidak saya kerjakan karena laporan ini fokus pada versi Breeze.
3. Belum ada automated test untuk fitur uploads. Verifikasi saya lakukan lewat browser, phpMyAdmin, dan pemeriksaan folder storage.
4. Menu Berkas hanya ditambahkan pada navigasi desktop. Versi responsive untuk menu mobile belum ada.
5. Server dijalankan pada port 8123, bukan 8000.
6. Gambar kode memakai versi yang komentarnya dirapikan. Komentar PHP dihapus dan beberapa baris panjang dipecah supaya tidak terpotong saat diekspor. Logika kode tidak berubah.

F. Kesimpulan

1. Migration dan model Upload berhasil dibuat. Tabel uploads terhubung ke tabel users lewat foreign key dengan cascade delete.
2. Resource controller dan resource route berhasil dipasang. Tujuh route uploads hanya bisa diakses setelah login.

3. CRUD berjalan, mulai dari unggah, daftar, detail, ubah, sampai hapus. File tersimpan di storage/app/public/uploads dan bisa dibuka lewat public/storage.
4. Validasi format dan ukuran berjalan. File .zip dan file 5,7 MB ditolak dengan pesan galat pada input file.
5. Data terpisah per pengguna. Berkas milik akun lain menghasilkan 403 Forbidden.
6. Pagination lima baris berjalan dan penomoran berlanjut di halaman kedua.
7. Ada dua keterbatasan yang saya temukan. File fisik tidak ikut terhapus saat akun dihapus lewat database, dan file di disk public bisa dibuka lewat URL tanpa login.

G. Link Repository Github

Kode program lengkap praktikum ini dapat diakses pada repositori GitHub berikut:

https://github.com/Dryhus21/PABW.git